

인공지능시스템

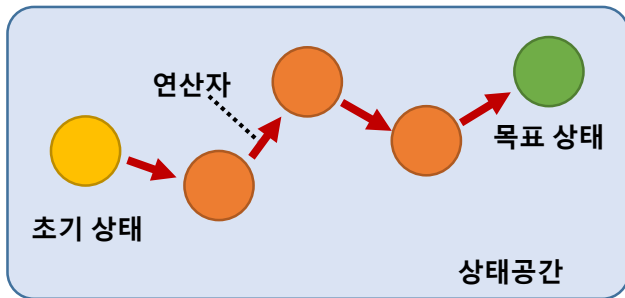
AI System

int**Ro**-Lab.

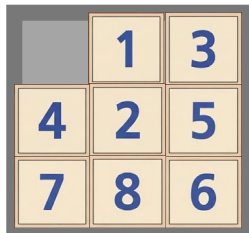
청주대학교 무인항공기학과
김상현 교수 (www.intro-lab.com)

◆ 탐색(Search)

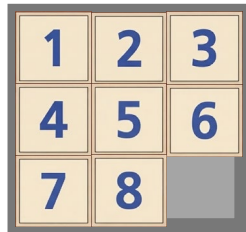
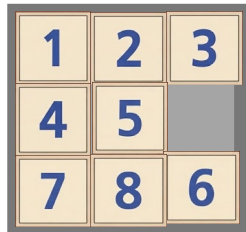
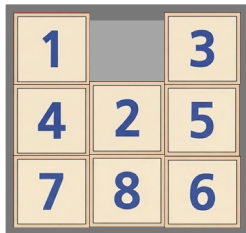
- 상태공간에서 시작상태에서 목표상태까지의 경로를 찾는 것
- 상태(State): 특정 시점의 상황을 나타내는 정보
 - 예시: 길찾기 문제의 경우에는 현재 "위치"가 "상태"임.
- 상태공간(State Space): 상태들이 모여있는 공간
 - 예시: 길찾기 문제의 경우에는 가능한 모든 위치 좌표.
- 연산자(Operator): 한 상태에서 다른 상태로 전이 시키기 위해 취할 수 있는 구체적인 행동
 - 예시: 길찾기 문제에서는 "어느 방향으로 이동하기"라는 행동



◆ 예시: 8-Puzzle

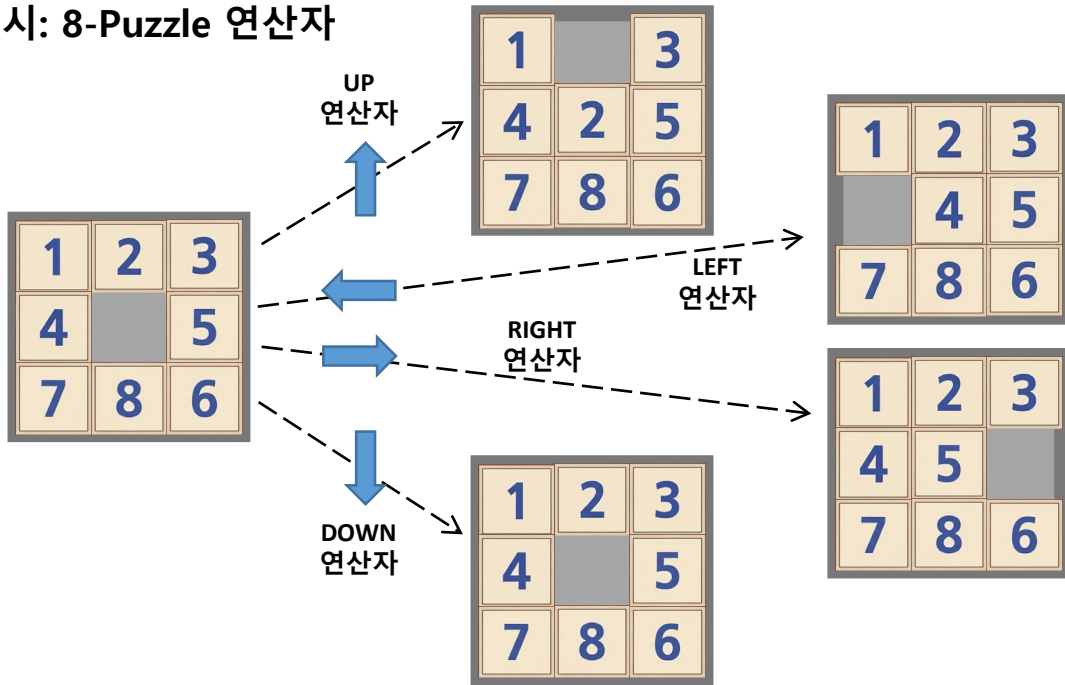


초기 상태



목표 상태

◆ 예시: 8-Puzzle 연산자

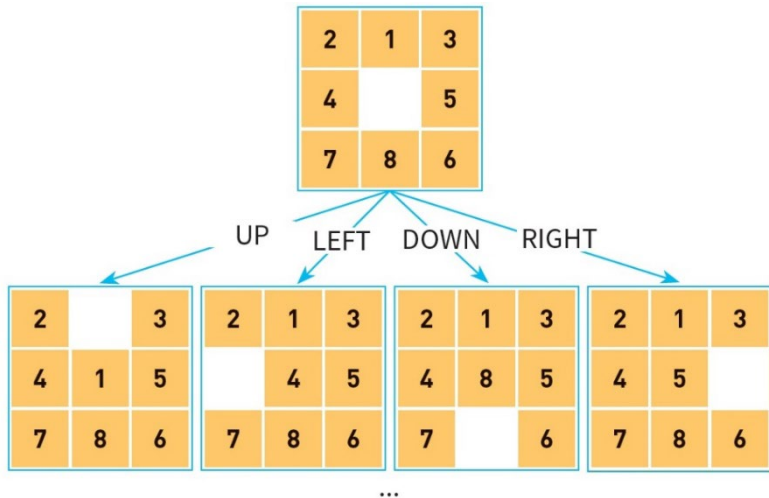


탐색 알고리즘

◆ 탐색 트리

- 상태 → 노드(Node)
- 초기 상태 → 루트 노드(Root Node)
- 연산자 → 간선(Edge)

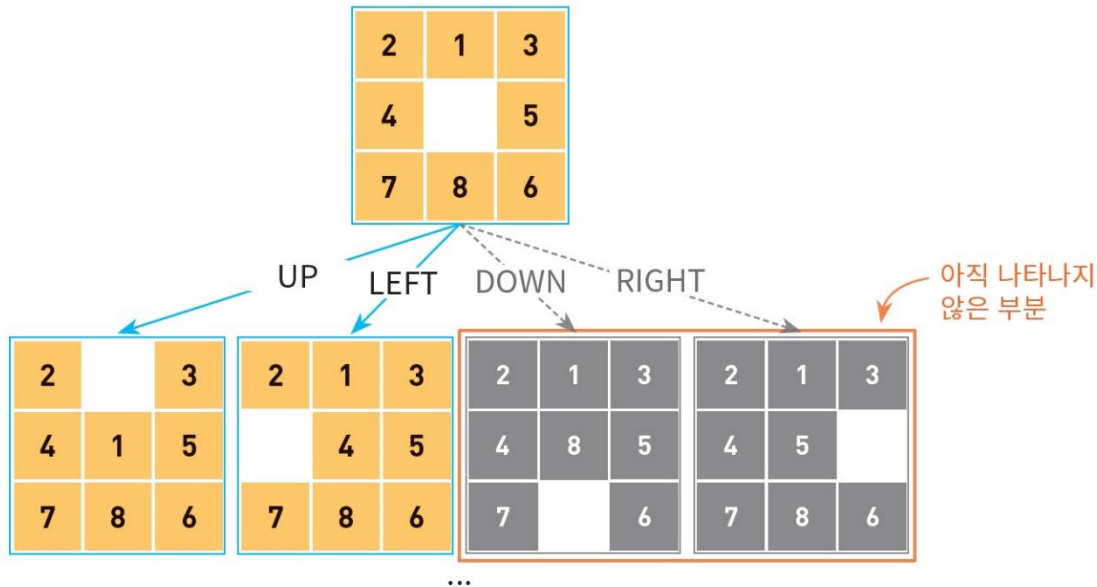
플레이어 수: 1명
단독 의사결정



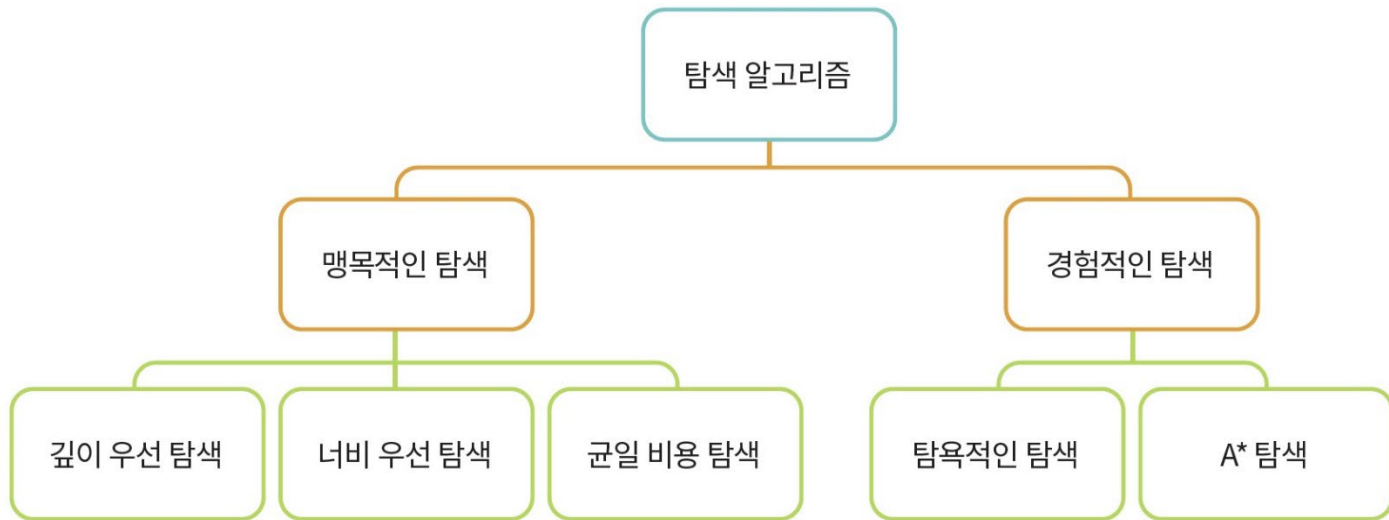
탐색 알고리즘

◆ 탐색 트리

- 연산자를 적용하기 전까지는 탐색 트리는 미리 만들어져 있지 않음.

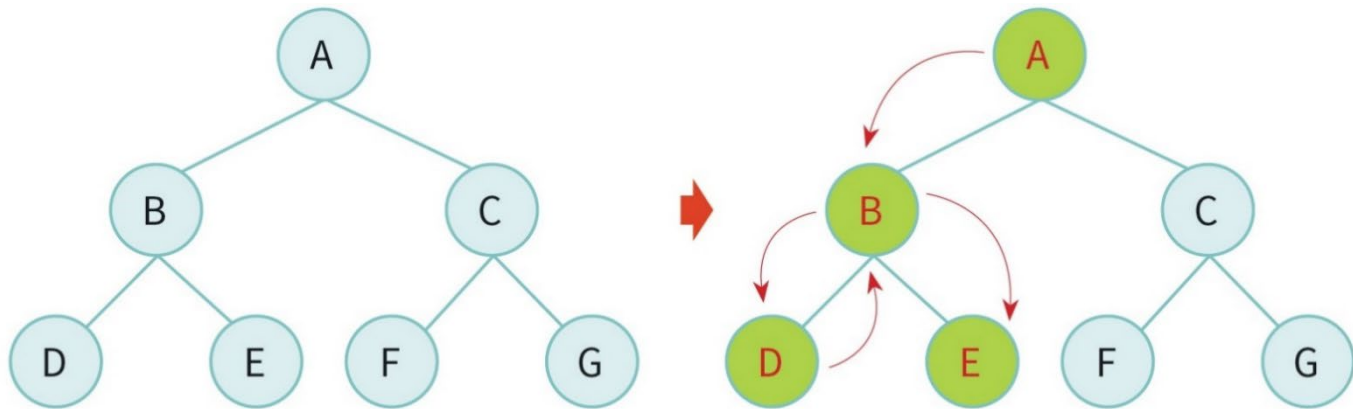


◆ 기본적인 탐색 기법



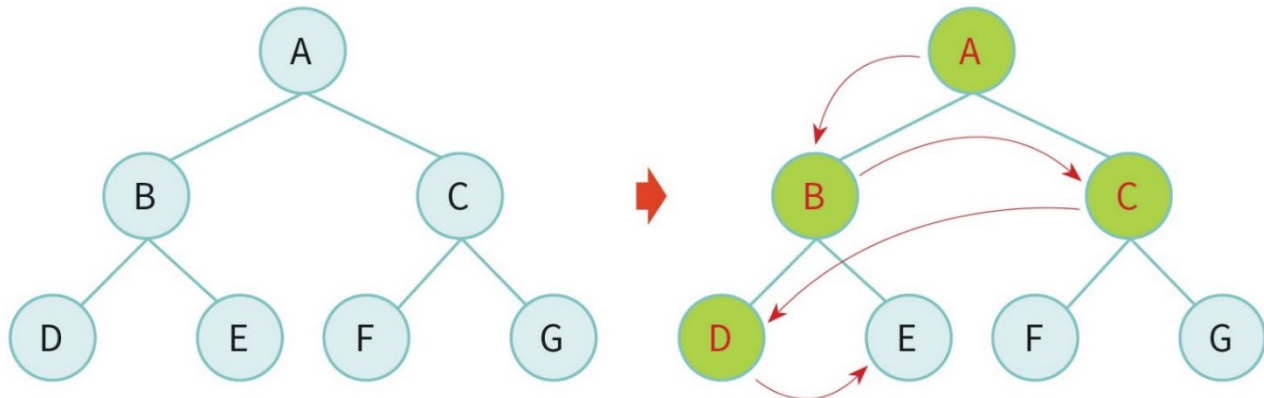
◆ 깊이 우선 탐색(DFS, Depth-First Search)

- 깊이 우선 탐색은 탐색 트리 상에서, 해가 존재할 가능성이 존재하는 한, 앞으로 계속 전진하여 탐색하는 방법이다.



◆ 너비 우선 탐색(BFS, Breadth-First Search)

- 깊이 우선 탐색(depth-first search)은 탐색 트리 상에서, 해가 존재할 가능성이 존재하는 한, 앞으로 계속 전진하여 탐색하는 방법이다.



◆ 열린 목록(Open List), 닫힌 목록(Closed List)

- 열린 목록: 앞으로 탐색할 후보 노드들을 모아둔 목록
 - 목적: 다음으로 탐색할 가장 좋은 노드를 선택
- 닫힌 목록: 이미 탐색을 완료한 노드들을 모아둔 목록
 - 동일한 노드를 중복 탐색하는 것을 방지

◆ 경험적인 탐색 방법

- 만약 우리가 문제 영역에 대한 정보나 지식을 사용할 수 있다면 탐색 작업을 훨씬 빠르게 할 수 있다. 이것을 경험적 탐색 방법(Heuristic Search Method) 또는 휴리스틱 탐색 방법이라고 부른다.
- 이때 사용되는 정보를 휴리스틱 정보(Heuristic Information)라고 한다.

◆ 8-Puzzle에서의 휴리스틱 → 탐욕적인 탐색(Greedy Search) 예시

- 예를 들어서 현재 상태와 목표 상태가 다음과 같다고 하자.

2	1	3
8	5	6
7	4	

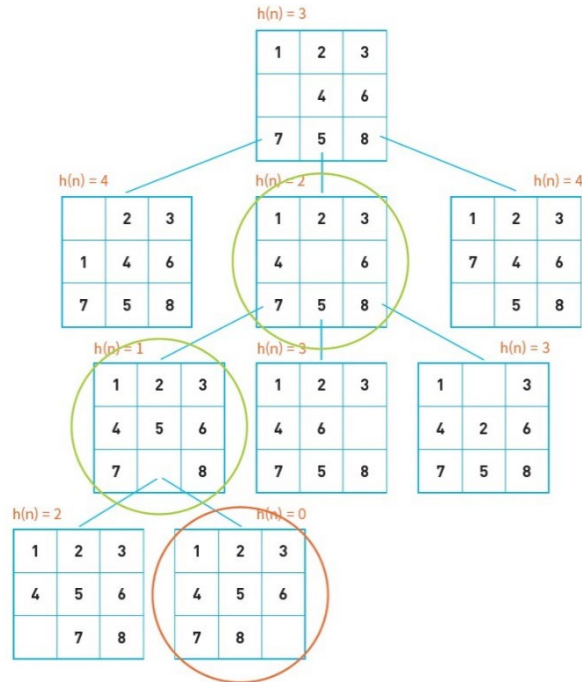
1	2	3
4	5	6
7	8	

- $h(n)$ = 현재 제 위치에 있지 않은 타일의 개수 = $1+1+1+1 = 4$

2	1	3
8	5	6
7	4	

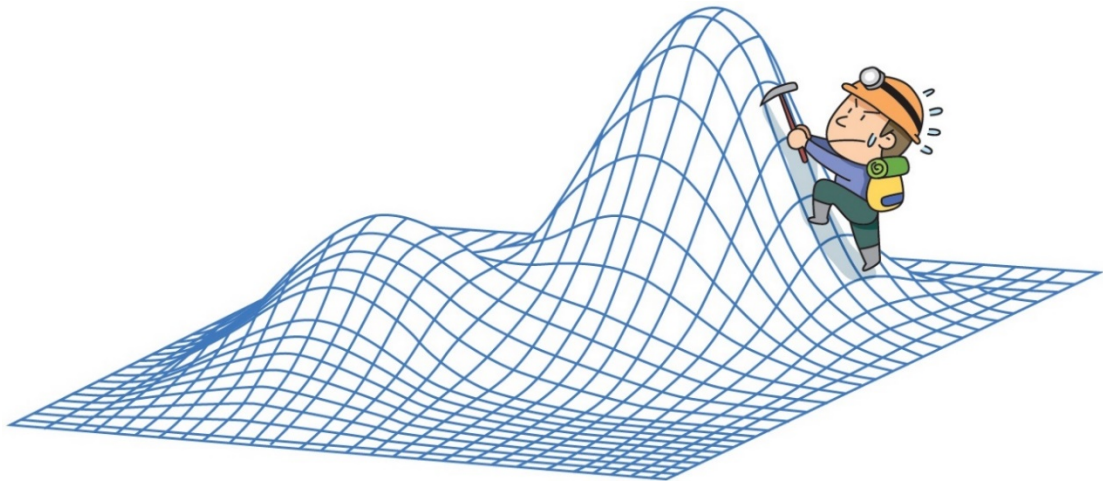
◆ 언덕 등반 기법

- 평가 함수로 제 위치에 있지 않은 타일의 개수 사용.
- 평가 함수의 값이 좋은 노드를 먼저 처리한다.



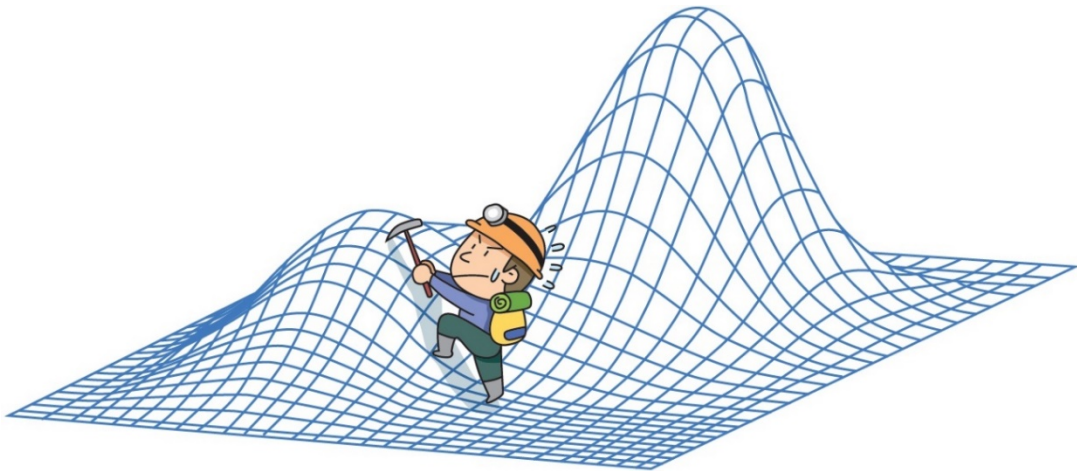
◆ 언덕 등반 기법

- 경험적인 탐색 방법은 무조건 휴리스틱 함수 값이 가장 좋은 노드만을 선택.
- 이것은 등산할 때 무조건 현재의 위치보다 높은 위치로만 이동하는 것과 같다.
- 일반적으로는 현재의 위치보다 높은 위치로 이동하면 산의 정상에 도달하는 것과 같은 이치.



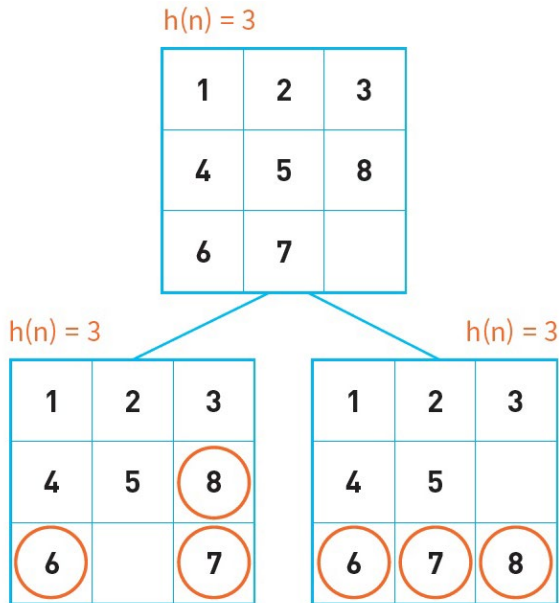
◆ 언덕 등반 기법: 지역 최소 문제(단점)

- 순수한 언덕 등반 기법은 오직 $h(n)$ 값만을 사용.
 - OPEN 리스트나 CLOSED 리스트도 사용하지 않음.
- 생성된 자식 노드의 평가 함수 값이 부모 노드보다 더 높거나 같은 경우가 나올 때, 정상에 도달하지 못하는 상태 → 지역 최소 문제(Local Minima Problem)



◆ 언덕 등반 기법: 지역 최소 문제(단점)

● 예시

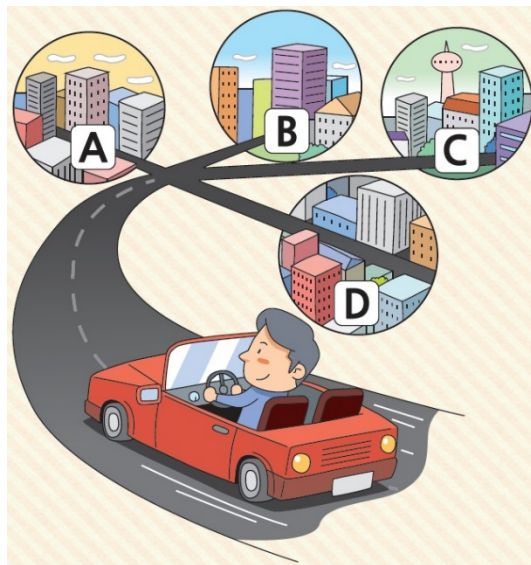


◆ Comparison of Pathfinding Algorithms

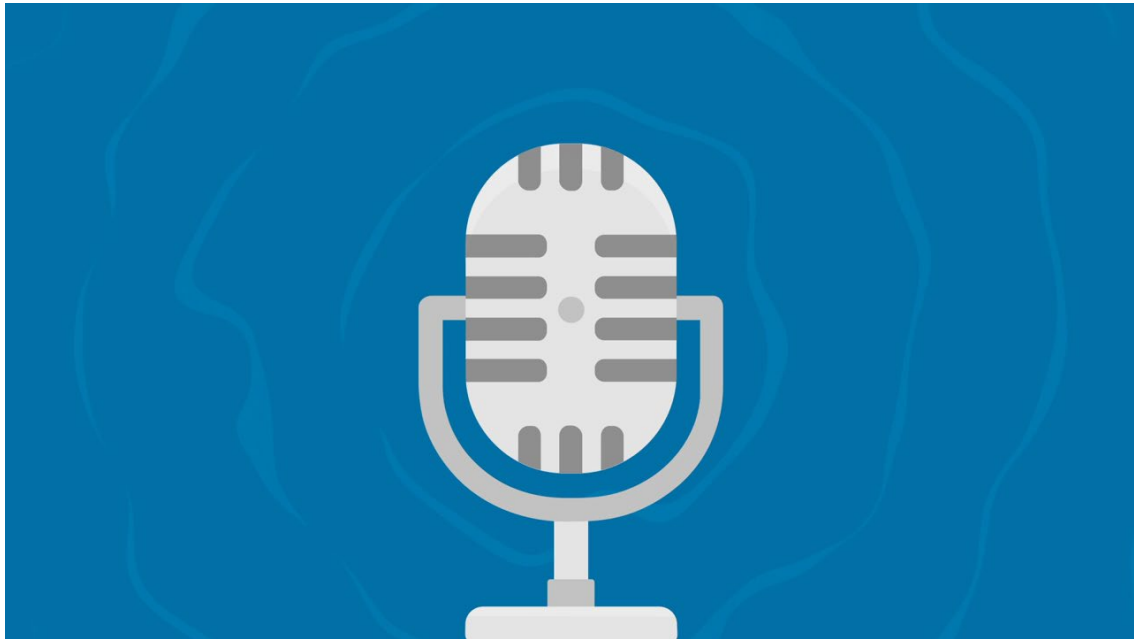


◆ 외판원 문제(TSP, Travelling Salesman Problem)

- 외판원 문제는 "도시의 목록과 도시들 사이의 거리가 주어졌을 때, 하나의 도시에서 출발하여 각 도시를 방문하는 최단 경로를 무엇인가?" 이다.



◆ 외판원 문제(TSP, Travelling Salesman Problem)



게임

◆ 게임 조건

- 두 명의 경기자: 연합 불가
- 제로섬 게임: 한 경기자의 승리는 곧 다른 경기자의 패배
- 차례대로 수를 두는 게임을 대상으로 함
- 예시: 바둑, 체스 등

플레이어 수: 2명
경쟁, 대결

◆ 게임을 인공지능에 도입한 이유

- 게임은 예전부터 인공지능의 매력적인 연구 주제였다.
- Tic-Tac-Toe나 체스, 바둑과 같은 게임은 추상적으로 정의할 수 있고 지적 능력과 연관이 있는 것으로 생각되었다.
- 이들 게임은 비교적 적은 수의 연산자들을 가진다. 연산의 결과는 엄밀한 규칙으로 정의된다.

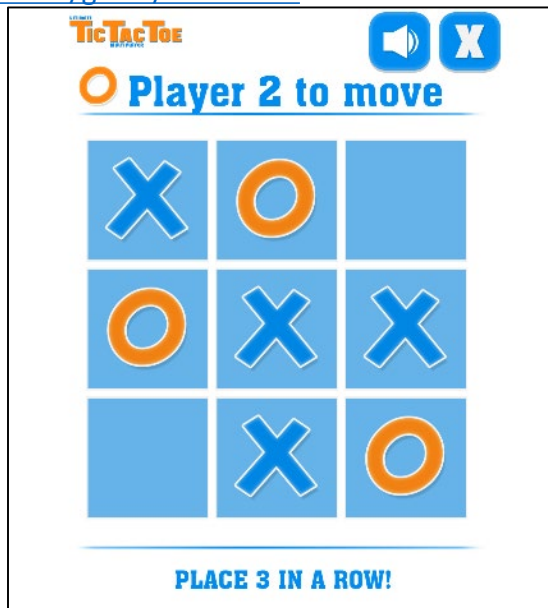


Tic-Tac-Toe

Photo by
Sang-Hyeon Kim

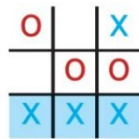
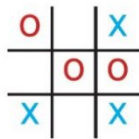
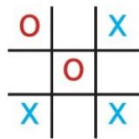
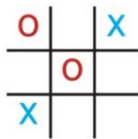
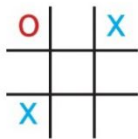
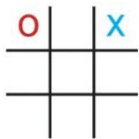
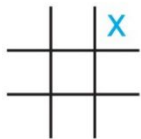
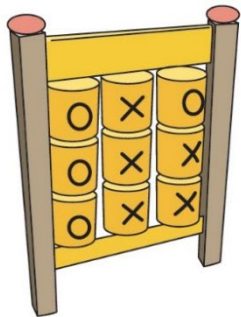
◆ Tic-Tac-Toe

- <https://www.crazygames.com/game/tic-tac-toe>

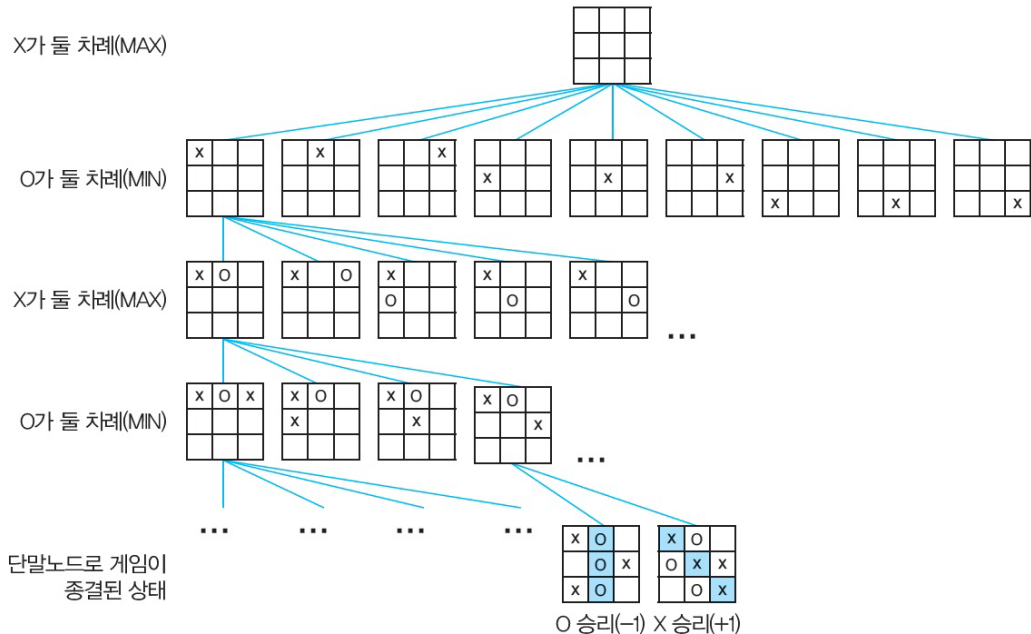


◆ 게임의 정의

- 2인용 게임
- 두 경기자를 MAX와 MIN으로 부름.
- 항상 MAX 경기자가 먼저 수를 둔다고 가정.



◆ Tic-Tac-Toe의 게임 트리



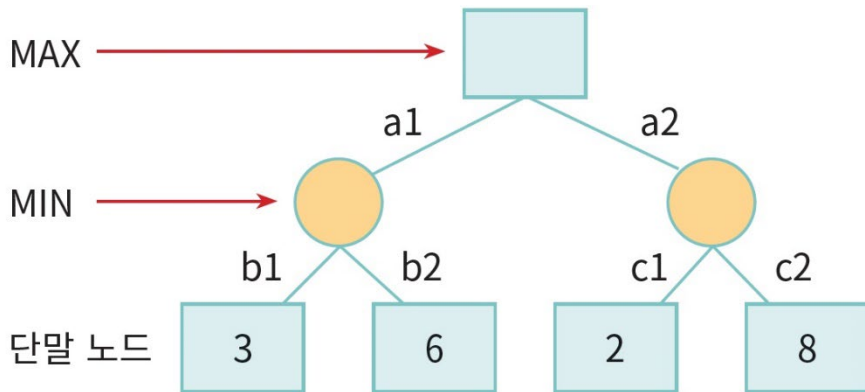
◆ Tic-Tac-Toe의 게임 트리의 크기

- Tic-Tac-Toe 게임 보드는 3×3 크기를 가지고 있고 한 곳에 수를 놓으면 다른 사람이 놓을 수 있는 곳은 하나가 줄어들게 된다.

$$9 \times 8 \times 7 \times \dots \times 1 = 9! = 362,880$$

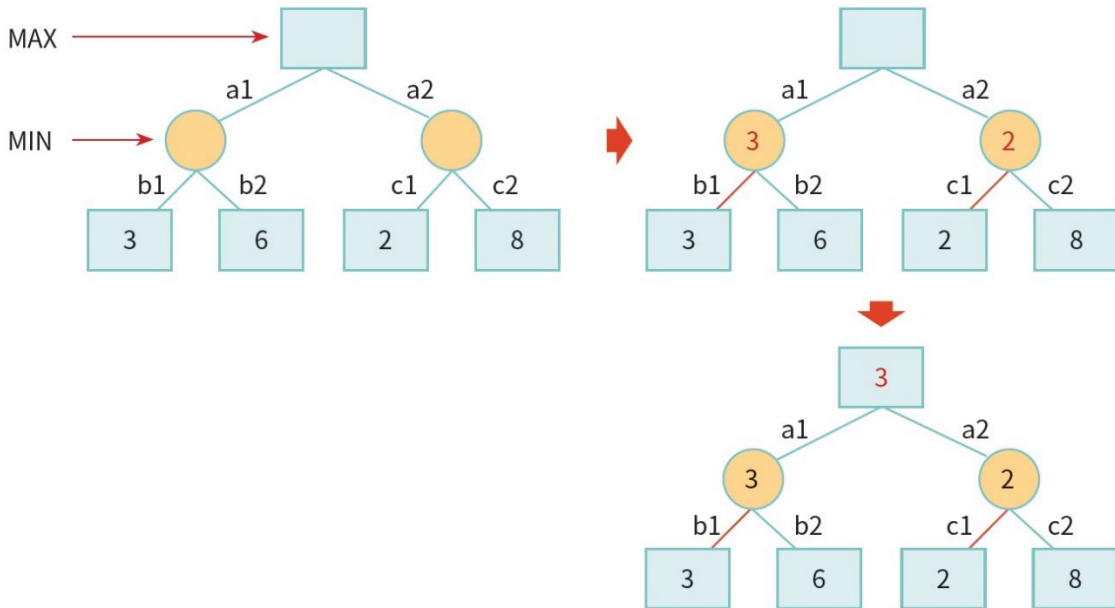
◆ 핵심

- 상대방이 최선의 수를 둔다고 가정하고, 현재 나의 최선의 수를 고민하는 것.

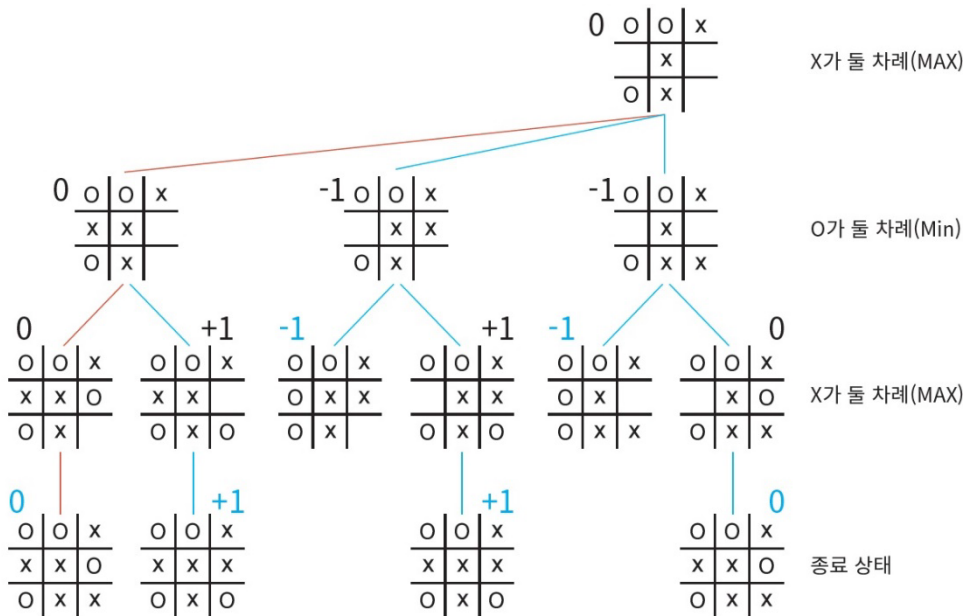


미니맥스 알고리즘

◆ 미니맥스(Minimax) 알고리즘 원리



◆ 틱택토(Tic-Tac-Toe) 게임에서의 미니맥스

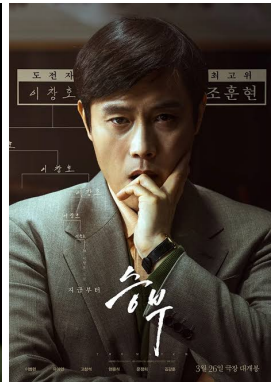


단말 노드 값

1. 무승부: 0
2. X(Max 승리): +1
3. O(Min 승리): -1

미니맥스 알고리즘

◆ 영화 승부 명장면: 수 읽기



◆ 영화 승부 명장면: 수 읽기

- 바둑은 이론적으로 미니맥스(Minimax) 알고리즘으로 푸는 것이 가능하지만,
- 현실적으로는 컴퓨터의 연산 능력 한계 때문에 불가능에 가깝습니다.

◆ 천문학적인 경우의 수 (상태 공간 복잡도)

- 틱택토: 약 10^5 개
- 체스: 약 10^{123} 개
- 바둑: 약 10^{170} 개 (규칙에 맞게 놓을 수 있는 모든 경우의 수를 의미함)
- 우주 전체의 원자 수: 10^{80} 개

◆ 실습

